

Credit Card Fraud Detection using Machine Learning

1. Problem

The growth of electronic payment systems has made credit cards one of the most widely used payment methods, but it has also created more opportunities for fraudulent activity, making fraud detection a critical challenge for financial institutions. While credit card fraud detection can be framed as a binary classification problem — legitimate versus fraudulent — it is complicated in practice by two factors: fraudulent transactions represent only a small fraction of all transactions, which can cause models to favour the majority class and appear accurate while missing most fraud; and individual transaction attributes alone often provide limited signal, since fraud is better revealed through patterns in transaction history, timing, and customer spending behaviour than through any single attribute in isolation.

Based on these considerations, this study develops a machine learning pipeline for credit card fraud detection that combines exploratory data analysis, data preprocessing, behavioural and temporal feature engineering, and supervised model evaluation, with the aim of providing a systematic approach to improving fraud detection performance and understanding the contribution of engineered features.

2. Data

2.1 Dataset Overview

The notebook uses 2 source files: Train (1,296,675 transactions) and Test (555,719 transactions), each with 23 raw columns. Each row represents a single card transaction, combining transaction-level attributes (timestamp, merchant, merchant category, amount, merchant coordinates) with cardholder attributes (name, gender, address, job, date of birth, card number) and the binary target, `is_fraud`. A column-level profile of the combined dataset confirmed zero missing values across all 23 raw columns, and the data types were consistent with each field's semantics. Duplicate rows were removed from the training file before feature engineering.

2.2 Key Features

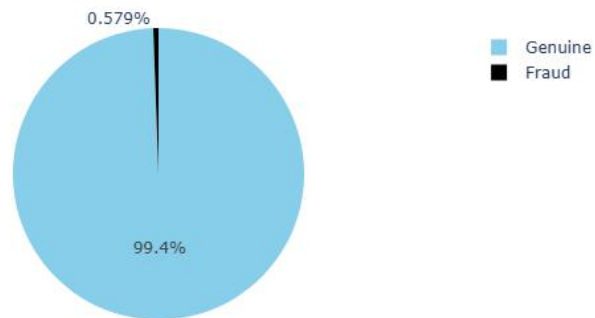
Features	Type	Description
amt	Existing feature	Transaction amount
city_pop	Existing feature	Population of the cardholder's city
category	Existing feature	Merchant category of the transaction
previous_transactions	Engineered feature	Number of previous transactions associated with the customer
time_since_last_transaction	Engineered feature	Time elapsed since the customer's previous transaction
customer_total_amount	Engineered feature	Cumulative transaction amount associated with the customer

customer_avg_amt_excluding_current	Engineered feature	Customer's average transaction amount calculated from previous transactions, excluding the current transaction
amt_to_avg_ratio	Engineered feature	Ratio of the current transaction amount to the customer's historical average transaction amount
distance	Engineered feature	Geographical distance between the cardholder and the merchant
hour	Engineered feature	Hour of the day when the transaction occurred
hour_sin, hour_cos	Engineered feature	Cyclic sine and cosine transformation of the transaction hour
age	Engineered feature	Cardholder's age at the time of the transaction

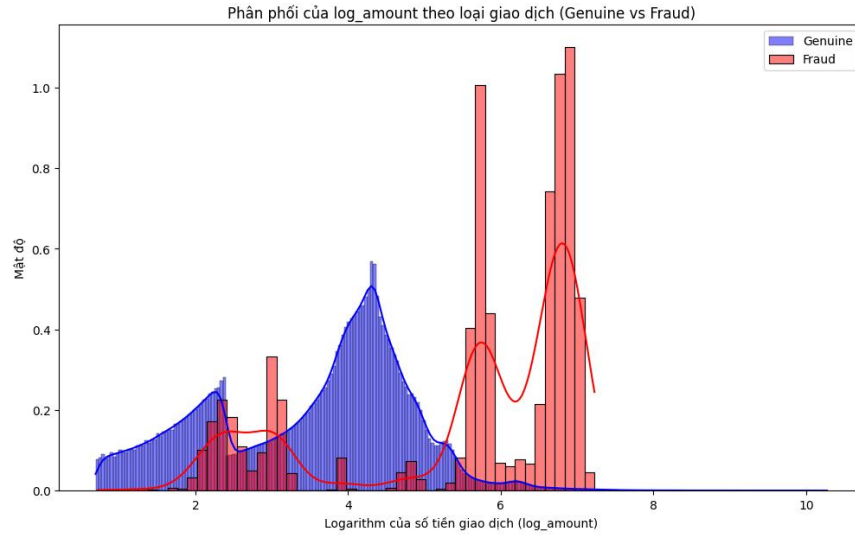
Table 1: Key features used for model training

2.3. EDA Findings

- Severe class imbalance: Out of the 1,296,675 Train transactions, only 7,506 (0.58%) were fraudulent — roughly one fraud for every 172 genuine transactions. This pattern is consistent with the severe imbalance typically reported for real-world transactional fraud data (Haixiang et al., 2017).



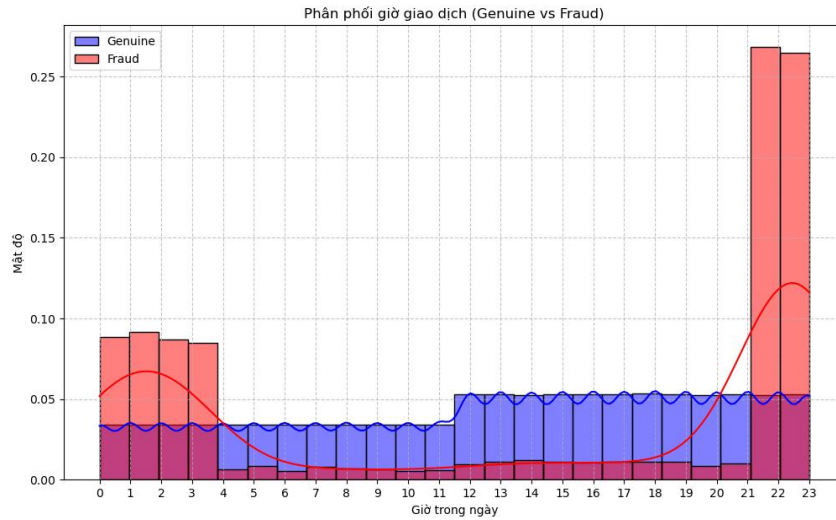
- Transaction amount: The log-transformed amount distribution differed visibly between genuine and fraudulent transactions, motivating the use of amount-based ratio.



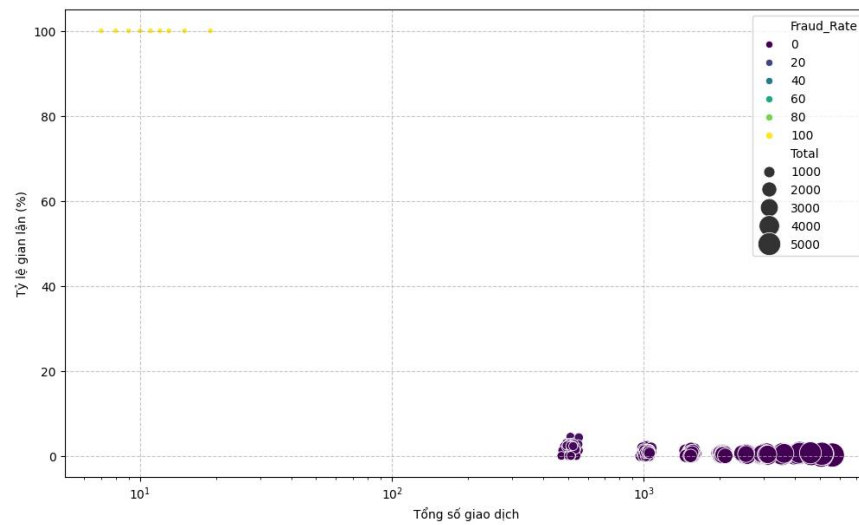
- Amount-to-average ratio: Binning amt_to_avg_ratio revealed a strong, monotonic relationship with fraud incidence (Table 1) — rising from 0.29% of transactions at ratio ≤ 0.5 to 23.39% of transactions at ratio > 8 . A transaction disproportionately larger than a card's own historical average is therefore one of the strongest behavioural indicators in the dataset.

Ratio bin	Transactions	Frauds	Fraud rate (%)
≤ 0.5	544,741	1,589	0.29
0.5 - 1.0	327,285	204	0.06
1.0 - 1.5	195,578	212	0.11
1.5 - 2.0	92,202	196	0.21
2.0 - 3.0	82,923	347	0.42
3.0 - 4.0	18,449	423	2.29
4.0 - 5.0	9,957	604	6.07
5.0 - 6.0	7,146	500	7.00
6.0 - 8.0	5,518	419	7.59
> 8.0	12,876	3,012	23.39

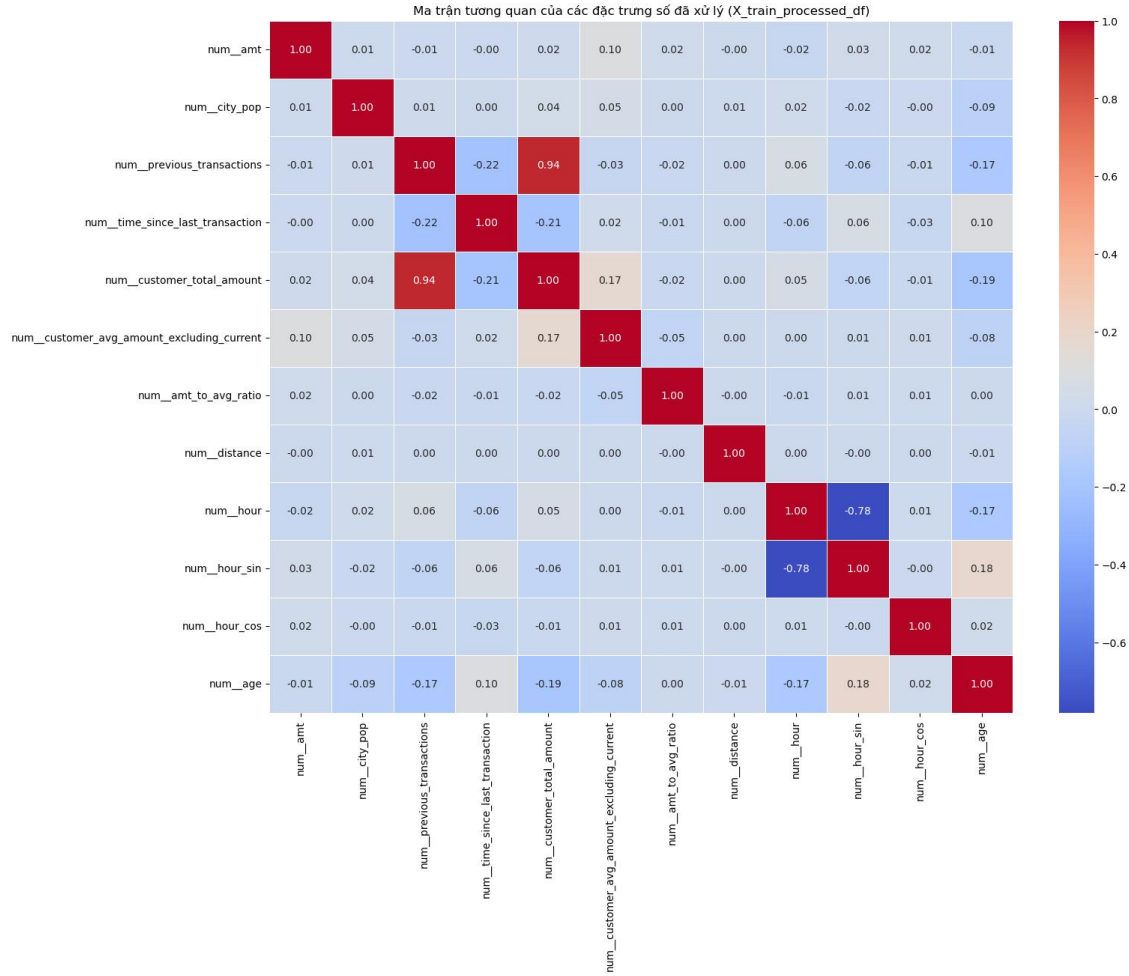
- Transaction time in the day: The distribution of transaction hour differed between genuine and fraudulent cases, motivating the cyclical sin/cos encoding used to represent the 24-hour periodicity for tree- and margin-based models.



- City-level fraud rate: A scatter analysis of raw per-city fraud rate against transaction volume showed that several low-volume cities exhibited a 100% raw fraud rate — an artefact of small sample size rather than genuine city risk — which motivated Bayesian smoothing of the exploratory city_fraud_encoded feature toward the overall fraud rate.



- Multicollinearity screening: a correlation heat map of the processed numeric predictors was produced to visually screen for redundancy among the twelve final numeric features before model training.



2.4. Evaluation metrics

Given a fraud incidence of only 0.58%, a naive classifier that never predicts fraud would already achieve roughly 99.4% accuracy while catching zero fraudulent transactions, which makes accuracy an uninformative metric for this problem. The notebook therefore evaluates models on precision, recall, F1-score and, critically, the F2-score, which weights recall twice as heavily as precision. This choice reflects the asymmetric cost structure of fraud detection: a missed fraud (false negative) causes a direct, often unrecoverable financial loss to the cardholder or issuer, whereas a false alarm (false positive) only costs the incremental effort of a manual review before being dismissed. Accordingly, the XGBoost decision threshold is deliberately selected by scanning a grid of probability cut-offs on the validation set and choosing the one that maximises F2 (Section 3.7).

3. Methodology

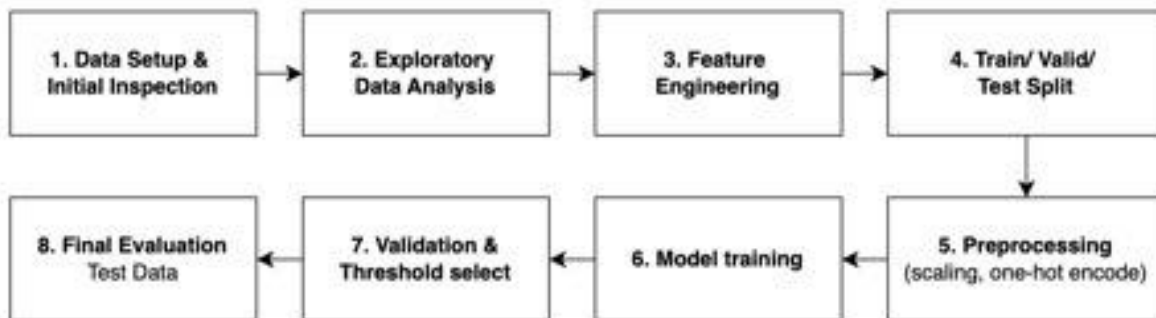


Figure 1. End-to-end pipeline used in the notebook.

3.1 Data Setup and Initial Inspection

Through initial structural inspection, no null values were detected, and duplicate observations were removed from the training set. To reduce the risk of information leakage and exclude non-informative identifiers, the row index and direct-identification fields (first, last, and trans_num) were excluded from the model input features

3.2 EDA

Univariate inspection covered the class balance of is_fraud, the distribution of transaction amount (raw and log-transformed), and the distribution of transaction hour. Bivariate analysis compared these distributions between genuine and fraudulent transactions and examined fraud rate as a function of amt_to_avg_ratio bins and of per-city transaction volume, as summarised in Section 2.4.

3.3 Feature Engineering and Rationale

Transactions were first sorted by card number and timestamp, after which two families of engineered features were constructed. Historical behavioral features (previous_transactions, time_since_last_transaction, amt_to_avg_ratio, customer_total_amount, customer_avg_amt_excluding_current) capture how a transaction compares with a card's own recent history - a sudden, unusually large purchase relative to a customer's own baseline is a well-established fraud signal. Contextual features (distance, hour/hour_sin/hour_cos, age) capture spatial anomalies, temporal patterns, and demographic context. Because the behavioural features are cumulative by construction, they were recomputed independently within each of the train, validation and test partitions rather than computed once on the full dataset, to prevent data leakage.

3.4 Train / Validation / Test Split

To respect the temporal nature of the fraud problem and avoid leakage from future transactions into training (Bergmeir & Benítez, 2012), trained data was sorted by timestamp and split 80/20 into a training set (1,037,340 transactions, 1st January 2019 – 6th March 2020) and a validation set (259,335 transactions, 6th March 2020 – 21st June 2020).

3.5 Preprocessing

A scikit-learn ColumnTransformer was fit exclusively on the training partition and then applied, without refitting, to the validation and test partitions. 12 numeric features - amt, city_pop, previous_transactions, time_since_last_transaction, customer_total_amount, customer_avg_amt_excluding_current,

amt_to_avg_ratio, distance, hour, hour_sin, hour_cos and age - were standardised. The single low-cardinality categorical feature retained for modelling, category (14 levels) was encoded using one-hot encoding. The resulting processed feature matrix contains 26 columns.

3.6 Model Training

Three classifiers were trained on the identical processed feature matrix:

- Logistic Regression (solver = 'liblinear', max_iter = 1,000, random_state = 42), a linear baseline (Hosmer et al., 2013), evaluated at the default 0.5 probability threshold.
- Random Forest (random_state = 42, n_jobs = -1, with other hyperparameters kept at scikit-learn defaults), an ensemble of bagged decision trees (Breiman, 2001), with the classification threshold subsequently optimised for F2-score.
- XGBoost (objective = 'binary:logistic', n_estimators = 100, max_depth = 6, learning_rate = 0.3, subsample = colsample_bytree = 1.0, eval_metric = 'logloss', random_state = 42, n_jobs = -1), with scale_pos_weight set to the negative-to-positive class ratio to address class imbalance and the classification threshold optimised for F2-score.

3.7 Validation and Threshold Selection

For Random Forest, predicted probabilities on the validation set were converted to binary predictions across a grid of thresholds from 0.11 to 0.81 in steps of 0.1. After inspection, the threshold of 0.21 was found to maximise F2 on the validation set and was therefore selected for use in all subsequent evaluation.

Threshold	Precision	Recall	F1	F2
0.11	0.9950	0.5440	0.9363	0.8183
0.21	0.9977	0.7598	0.8927	0.8625
0.31	0.9983	0.8602	0.8563	0.8571
0.41	0.9985	0.9110	0.8251	0.8410
0.51	0.9985	0.9466	0.7952	0.8215
0.61	0.9984	0.9755	0.7516	0.7878
0.71	0.9981	0.9879	0.6886	0.7330
0.81	0.9975	0.9923	0.5858	0.6381

For XGBoost, predicted probabilities on the validation set were converted to binary predictions across a grid of thresholds from 0.11 to 0.91 in steps of 0.1. After inspection, the threshold of 0.91 was found to maximise F2 on the validation set; therefore, selected for use in all subsequent evaluation.

Threshold	Precision	Recall	F1	F2
-----------	-----------	--------	----	----

0.11	0.260	0.989	0.412	0.634
0.31	0.402	0.978	0.570	0.760
0.51	0.513	0.967	0.670	0.822
0.71	0.626	0.947	0.754	0.859
0.91 (selected)	0.810	0.915	0.859	0.892

3.8 Final Evaluation on the Independent Test Set

Using the validation-selected threshold of 0.91, the trained XGBoost model was applied to the Test dataset for final evaluation. A 100-iteration bootstrap resample of the test set was additionally performed to quantify the sampling variability of precision, recall and F2.

4. Results

4.1 Validation Performance Across Models

Table 3 compares the three classifiers at the operating point actually used for each: Logistic regression baselines at the default 0.5 threshold, Random forest at optimal XGBoost at its F2-optimal threshold of 0.91.

Model	Precision	Recall	F1-score	F2-score
Logistic Regression	0.7535	0.2802	0.4085	0.3204
Random Forest	0.7456	0.8966	0.8143	0.8618
XGBoost	0.8105	0.9148	0.8595	0.8919

Table 3. Model Comparison on the Validation Set

Logistic Regression substantially under-detects fraud at the default threshold (recall = 0.28 on the class-1 report, F2 = 0.320), reflecting the well-known difficulty of an unweighted linear classifier in separating a 0.58%-prevalence minority class. Random Forest performs considerably better at the same default threshold (F1 = 0.814, F2 = 0.862). Evaluated at its F2-optimal threshold, XGBoost achieves moderately lower F1 to Random Forest but a materially higher F2 (0.892), making XGBoost the pipeline's selected final model.

4.2 Confusion Matrices and Final Metrics

Set	True Negatives	False Positives	False Negatives	True Positives
Validation (n = 259,335)	257,468	329	131	1,407
Test (n = 555,719)	552,904	670	262	1,883

Table 4. XGBoost Confusion Matrices (Threshold = 0.91)

Metric	Validation	Test
Precision	0.810	0.738
Recall	0.915	0.878
F1-score	0.859	0.802
F2-score	0.892	0.846

Table 5. XGBoost Performance: Validation vs Independent Test Set

On the independent test set, XGBoost correctly identified 1,883 of the 2,145 actual frauds (recall = 0.878) while flagging 670 legitimate transactions as false positives (precision = 0.738). A 100-iteration bootstrap resample of the test set confirmed this performance is stable rather than a sampling artefact: mean F2 = 0.846 (SD = 0.007), mean precision = 0.738 (SD = 0.009), mean recall = 0.878 (SD = 0.007), with 95% of resamples falling within a narrow band (F2 range across the 100 resamples: 0.828 – 0.860).

Metric	Mean	Std. Dev.	Min	25%	Median	75%	Max
F2-score	0.736	0.006	0.721	0.732	0.736	0.740	0.750
Precision	0.411	0.007	0.394	0.407	0.411	0.416	0.425
Recall	0.917	0.006	0.899	0.913	0.917	0.921	0.930

Although there is a moderate decline performance between validation and test (F2: 0.892 → 0.846; precision: 0.810 → 0.738), the bootstrap results show that test-set performance was relatively stable, with small standard deviations for F2, precision, and recall, suggesting that the observed decline is unlikely to be explained solely by random variation.

4.3 Feature Importance

Rank	Feature	Importance
1	amt	0.169
2	hour_cos	0.148
3	category = gas_transport	0.115
4	category = food_dining	0.067
5	category = shopping_pos	0.054
6	category = shopping_net	0.052

7	category = grocery_net	0.048
8	category = home	0.047
9	category = grocery_pos	0.044
10	category = entertainment	0.041

Table 6. Top 10 XGBoost Feature Importances (Gain-based)

The transaction amount (amt, importance = 0.169) and the cyclical hour encoding (hour_cos, 0.148) are the most influential predictors, followed by several merchant-category indicators, led by gas_transport (0.115). Among the remaining engineered predictors, the raw hour (0.019), time_since_last_transaction (0.012), customer_avg_amt_excluding_current (0.010), hour_sin (0.009) and age (0.008) contribute smaller but non-trivial shares. Overall, the feature importance results indicate that, within the final 26-feature model, transaction size, time-of-day and merchant category are the primary drivers of the model's predictions, while the behavioural recency features provide a secondary but complementary signal.

4.4. Practical Feasibility

To assess the model's deployment feasibility in real-world financial systems, where strict response-time requirements may demand sub-1.5 ms latency per transaction, the execution time of the proposed model was evaluated on the test set (555,719 observations and 23 features).

The results demonstrate strong computational efficiency. Training and validation were completed in 18.4 seconds on a dataset containing 1,296,675 observations, indicating a relatively low computational cost during model development. For inference the model processed the entire test set in 1.6 seconds, corresponding to an average latency of approximately 0.00288 ms per transaction (2.88 μ s).

This inference speed is approximately 520 times faster than the stringent 1.5 ms latency requirement. These results therefore demonstrate the high computational feasibility of the proposed framework for integration into real-time transaction processing streams, where rapid fraud detection is essential without introducing substantial processing delays.

5. Discussion and Implications

This paper conducted a comparative evaluation of three learning algorithms - Logistic Regression, Random Forests, and XG Boost—for detecting credit card fraud using a highly imbalanced, imulated credit card transaction dataset. The findings on training data show that while Random Forest demonstrated the highest precision (81.9%%), XG Boost achieved the highest recall and F2 score (89.2%), making it the most effective at detecting fraudulent transactions among tested models. From a practical perspective, the choice of algorithm should depend on institutional priorities. If the goal is to maximise fraud detection while maintaining a balance between recall and precision. XGBoost may be more suitable. If reducing false positives is critical—such as when the cost of false alerts is particularly high—Random Forest provides the strongest performance. By clarifying these trade-offs, this study not only contributes to the academic discussion but also offers actionable guidance for financial institutions in selecting the most appropriate algorithm for fraud detection.

The results also highlight the fundamental trade-off between the detection rate (TPR) and false-positive rate (FPR) in fraud detection, particularly under severe class imbalance. Since increasing TPR inevitably generates more false alarms, the model would be better deployed through a review-and-escalation workflow rather than as a fully automated transaction-blocking system. This approach is justified because even a low false-positive rate can generate substantial customer friction and review costs at scale. At the same time, the model's high recall concentrates genuine fraud cases in the review queue, enabling analysts to focus on a smaller, higher quality set of transactions. The decision threshold should also be selected according to the institution's tolerance for fraud loss, customer friction, and review capacity. Institutions with particularly high average fraud transaction values or low review queue capacity may find that a lower threshold (higher recall, lower precision) better minimizes total expected cost, whereas those with strict customer experience mandates may prefer a higher threshold.

6. Limitations and Recommendations for Improvement

Despite its promising results, this study has two main limitations. First, SMOTE was not applied to address the severe class imbalance. Although F2-score and threshold optimization were used to prioritize fraud recall, resampling techniques such as SMOTE could be explored in future studies to assess whether improving minority-class representation enhances model performance. Second, model evaluation focused primarily on F2-score, precision, recall, and F1-score, without considering PR-AUC and ROC-AUC. While F2-score is appropriate for prioritizing fraud recall, it provides a threshold-specific assessment. Future research should incorporate PR-AUC and ROC-AUC to provide a more comprehensive evaluation across different classification thresholds. PR-AUC is particularly relevant for highly imbalanced fraud detection, as it better reflects the precision–recall trade-off for the minority class. Future studies could therefore compare different imbalance-handling techniques and evaluation metrics, while also incorporating cost-sensitive analysis and systematic hyper-parameter optimization to improve the robustness and practical applicability of the proposed framework.

7. Conclusion

This study developed a machine-learning framework for credit card fraud detection using behavioural feature engineering and three classification models: Logistic Regression, Random Forest, and XGBoost. Given the severe class imbalance, F2-score was adopted as the primary optimisation criterion, placing greater emphasis on recall. The results showed that XGBoost achieved the strongest overall performance, with a validation F2-score of 0.8919, compared with 0.8618 for Random Forest and 0.3205 for Logistic Regression. Overall, the findings suggest that XGBoost, combined with behavioural feature engineering and threshold optimisation, provides an effective approach to transaction-level fraud detection. Future research incorporating SMOTE, PR-AUC, ROC-AUC, and cost-sensitive evaluation could further strengthen the robustness and comprehensiveness of the proposed framework.